

ARTIFICIAL INTELLIGENCE: CONCEPTS AND TECHNIQUES

WEEK 10:

Lecture 45: An Introduction to Deep Learning

This document, "**Lecture 45: An introduction to Deep Learning**," by V. Susheela Devi, NPTEL, introduces deep learning and contrasts it with traditional machine learning approaches.

Here's a summary:

The document begins by explaining **Feature Engineering (FE)**, which involves manually transforming raw data into features for predictive models. It highlights that FE is crucial for model performance and is often considered an art.

It then introduces **Deep Learning (DL)** as a family of methods that use deep architectures to learn high-level feature representations automatically, reducing the need for manual feature engineering.

Key aspects of Deep Learning covered include:

- **Definitions:**

Multiple definitions are provided, generally emphasizing deep architectures with many layers of information processing for feature extraction, transformation, and pattern analysis.

- **DL Basics:**

It defines terms like Deep Belief Networks (DBN), Boltzmann Machines (BM), Restricted Boltzmann Machines (RBM), Deep Neural Networks (DNN), and Deep Auto-encoders.

- **Evolution of DL:**

It explains that while multilayer neural networks have existed for a long time, what's new in deep learning are algorithms that effectively train networks with many hidden layers.

- **Characteristics of DL:**

DL networks automatically choose required features, and multiple layers uncover hierarchical features. A solution to training many layers is to train one layer at a time.

- **New Training Method:**

The document details a layer-wise pre-training approach where each non-output layer is trained as an auto-encoder to learn good features from the previous layer's output. This process involves training one layer, then fixing it and training the next, and so on, before a final fine-tuning step using backpropagation.

- **Architectures:**

It lists common deep network architectures such as Deep Belief Networks (DBN), Recurrent Neural Networks (RNN) (including Long Short-Term Memory - LSTM), and Convolutional Neural Networks (CNN).

- **Categorization:**

Deep learning networks are categorized into:

- **Unsupervised or Generative Learning:**

For capturing high-order correlations without target labels.

- **Supervised Learning:**

For pattern classification with target labels.

- **Hybrid Deep Networks:**

Combining generative and supervised approaches for improved discrimination.

- **Two-way Classification Scheme:**

This scheme further divides models into:

- **Deep Discriminative Models:**

(DNN, RNN, CNN) efficient, flexible, and suitable for end-to-end learning.

- **Generative Unsupervised Models:**

(RBM, DBN, DBM, Regularized autoencoders) easier to interpret, embed domain knowledge, compose, and handle uncertainty.

- **Applications in NLP:**

The lecture concludes by mentioning tasks in Natural Language Processing (NLP) that utilize deep learning, such as text classification, language modeling, machine translation, document summarization, and question answering.

Lecture 46

This file, "**Lecture 46: Datasets for Deep Learning and CNN,**" by V. Susheela Devi for NPTEL, covers various datasets used in deep learning, particularly for image, speech, and natural language processing (NLP), and introduces the architecture and components of Convolutional Neural Networks (CNNs).

Key topics include:

- **Datasets for Deep Learning:**

- **Image Datasets:**

ImageNet (14 million labeled images across 2000+ categories), MNIST (handwritten digits, 60,000 training, 10,000 test examples), CIFAR-10 (32x32 pixel images of 10 object classes, 60,000 images), Microsoft COCO (2.5 million object instances in 328K images for segmentation), and VQA (Visual Question Answering dataset with images, questions, and answers).

○ **NLP Datasets:**

IMDB dataset (25,000 movie reviews for training and testing for binary sentiment classification), Twenty Newsgroups (20,000 messages from 20 newsgroups), Sentiment 140 (160,000 tweets for sentiment analysis), WordNet (117,000 English synsets), and The Wikipedia Corpus (1.9 billion words from 4.4 million articles).

○ **Speech Datasets:**

TIMIT (speech transcription), VoxCeleb (celebrity voice transcription), WMT '15 (English-Czech translation), WMT/14 (English-German translation), IWSLT '15 (English-Vietnamese translation), CHIME 5 (multiple speaker speech recognition), and LRS3-TED (visual speech recognition).

● **Turning Text into Features:**

Discusses methods to convert text into numerical features for neural networks, including representing words as integers, one-hot encoding, and bag-of-words.

● **Word Embeddings:**

Explains word embeddings as vector representations of words learned from context, enabling comparison and visualization of semantic relationships. Techniques like Word2vec, GloVe, and fastText are mentioned.

● **Batch Normalization:**

Describes this technique to standardize and normalize the input of a layer from a previous layer, using mean, standard deviation, and learnable parameters.

● **Convolutional Neural Networks (CNNs):**

○ **Concept:**

Introduces CNNs as a type of smaller neural network architecture that arranges neurons in three dimensions (width, height, depth) and is effective for image processing.

○ **Architecture:**

Comprises two main parts: Feature Extraction (convolution and pooling) and Classification (fully connected layers).

● **Layers in CNN:**

■ **Convolution Layer:**

Extracts features by performing a mathematical operation between the input image and a filter, producing a feature map.

■ **Pooling Layer:**

Decreases the size of the convolved feature map to reduce computational costs. Types include Max Pooling, Average Pooling, and Sum Pooling.

■ **Fully Connected Layer:**

Connects neurons between layers, flattens the input from previous layers, and performs classification.

- **Parameters:**

- **Dropout:**

A technique to prevent overfitting by randomly dropping neurons during training.

- **Activation Functions:**

Functions like ReLU, Softmax, tanH, and Sigmoid, which introduce non-linearity to the network and decide which information to propagate.

Lecture 47

This document, "**Lecture 47: CNN and Introduction to RNN**," provides an overview of various Convolutional Neural Network (CNN) architectures and an introduction to Recurrent Neural Networks (RNNs).

Key topics covered on CNNs include:

- **Convolutional Layers and Filters:**

Explains how convolutional layers use filters to perform operations and detect small patterns in input images (e.g., 3x3 filters on a 6x6 image). It illustrates how stride affects the output and how filters operate on color images with multiple channels (RGB).

- **Max Pooling:**

Describes max pooling as a method to reduce image size and create a "new image but smaller."

- **Comparison of CNN Architectures:**

- **LeNet-5 (1998):**

One of the earliest and simplest, with 2 convolutional and 3 fully connected (FC) layers, using average pooling and about 60,000 parameters.

- **AlexNet (2012):**

Deeper than LeNet-5 (8 layers), known for being one of the largest CNNs at the time, and the first to use ReLU activation and dropout.

- **VGG-16 (2014):**

Even deeper (13 convolutional and 3 FC layers), using smaller 2x2 and 3x3 filters, with 138 million parameters. VGG-19 is also mentioned.

- **GoogLeNet (Inception-v1, 2014):**

Introduced "Inception Modules" that use parallel convolutions with different filter sizes and 1x1 convolutions for dimensionality reduction. It has a 22-layer architecture with 5 million parameters.

- **Inception-v3 (2015):**

A successor to Inception-v1 with 24 million parameters, featuring factorisation methods (asymmetric 1xn and nx1 convolutions, 5x5 into two 3x3 convolutions) and the first to use batch normalization.

- **ResNet-50 (2015):**

Popularized "skip connections" to build deeper models and was among the first to use batch normalization. It has 26 million parameters.

- **Xception (2016):**

An adaptation of Inception that replaces Inception modules with depthwise separable convolutions, with similar parameters to Inception-v1 (23 million).

- **Inception-v4 (2016):**

From Google, with 43 million parameters, making changes to the Stem group and Inception-C module, and using uniform choices for Inception blocks.

- **Inception-ResNet-v2 (2016):**

Features 56 million parameters and converts Inception modules into Residual Inception blocks, with additional Inception modules.

- **ResNeXt-50 (2017):**

Includes parallel towers/branches within each module, scaling up the number of parallel towers. It has 25 million parameters.

- **DenseNet:**

Uses skip connections extensively, concatenating feature maps instead of adding them, leading to feature re-use and fewer parameters.

- **Bit Transfer (BiT) (2020):**

A scalable ResNet-based model, with larger variants pre-trained on massive datasets. It replaces batch normalization with group normalization and weight standardization.

- **Performance Comparison:**

Includes tables comparing various models based on the number of parameters, ImageNet Top-1 accuracy, Top-5 accuracy, and model size.

Introduction to Recurrent Neural Networks (RNNs):

- **Principle of RNN:**

Explains that in an RNN, the output from the previous step is fed as input to the current step, allowing the network to process sequences.

- **Complexity Reduction:**

Highlights that RNNs reduce complexity by using a single function f regardless of the input/output sequence length, acting as a compression from fully connected networks. The hidden representation s_i and output y_i are calculated using specific equations involving weights U , W , V , and biases b , c .

Lecture 48

This file, "**lecture48.pdf**", provides an overview of Recurrent Neural Networks (RNNs) and Generative Adversarial Networks (GANs).

RNNs

- **Types of RNNs:**

Deep RNN, Bidirectional RNN, and Pyramid RNN are discussed.

- **Naïve RNNs:**

The basic function $h', y = f(h, x)$ is introduced, along with the computation of h' and Y_i .

- **Applications of RNNs:**

Examples include one-to-one, one-to-many (e.g., image captioning), many-to-one (e.g., sentiment classification), and many-to-many (e.g., machine translation, video classification on a frame level).

- **Problems with Naïve RNNs:**

Forgetting old information in time series and the vanishing gradient problem are highlighted.

- **LSTM (Long Short-Term Memory):**

Introduced as a solution to RNN problems, emphasizing its cell state and gates (forget, input, output). The steps in an LSTM's operation are detailed.

- **GRU (Gated Recurrent Unit):**

A simpler variant of LSTM that combines the forget and input gates into a single update gate and merges the cell state and hidden state. It is noted to be speedier to train than LSTMs.

- **Stacked LSTM and Grid-LSTM:**

These architectures are mentioned for more complex RNN setups.

GANs

- **Concept:**

GANs involve adversarial training with two main components: a Generator (creates fake samples) and a Discriminator (distinguishes between real and fake samples). They are trained against each other in a zero-sum game.

- **Architecture:**

A latent random variable (noise) Z is fed into the Generator G to produce $G(z)$. The Discriminator D then receives both real images X and generated images $G(z)$ to classify them as real or fake.

- **Training Process:**

The discriminator is updated to improve its classification, and the generator is updated based on how well it fools the discriminator. The goal is for the generator to produce samples indistinguishable from real data.

- **GANs and CNNs:**

GANs typically use Convolutional Neural Networks (CNNs) for both the generator and discriminator, particularly in computer vision tasks due to their effectiveness with image data.

- **Types of GANs:**

Various types are listed, including Conditional GANs (CGANs), Deep Convolution Generative Adversarial Networks (DCGANs), and Wasserstein GANs (WGANs).

- **Applications of GANs:**

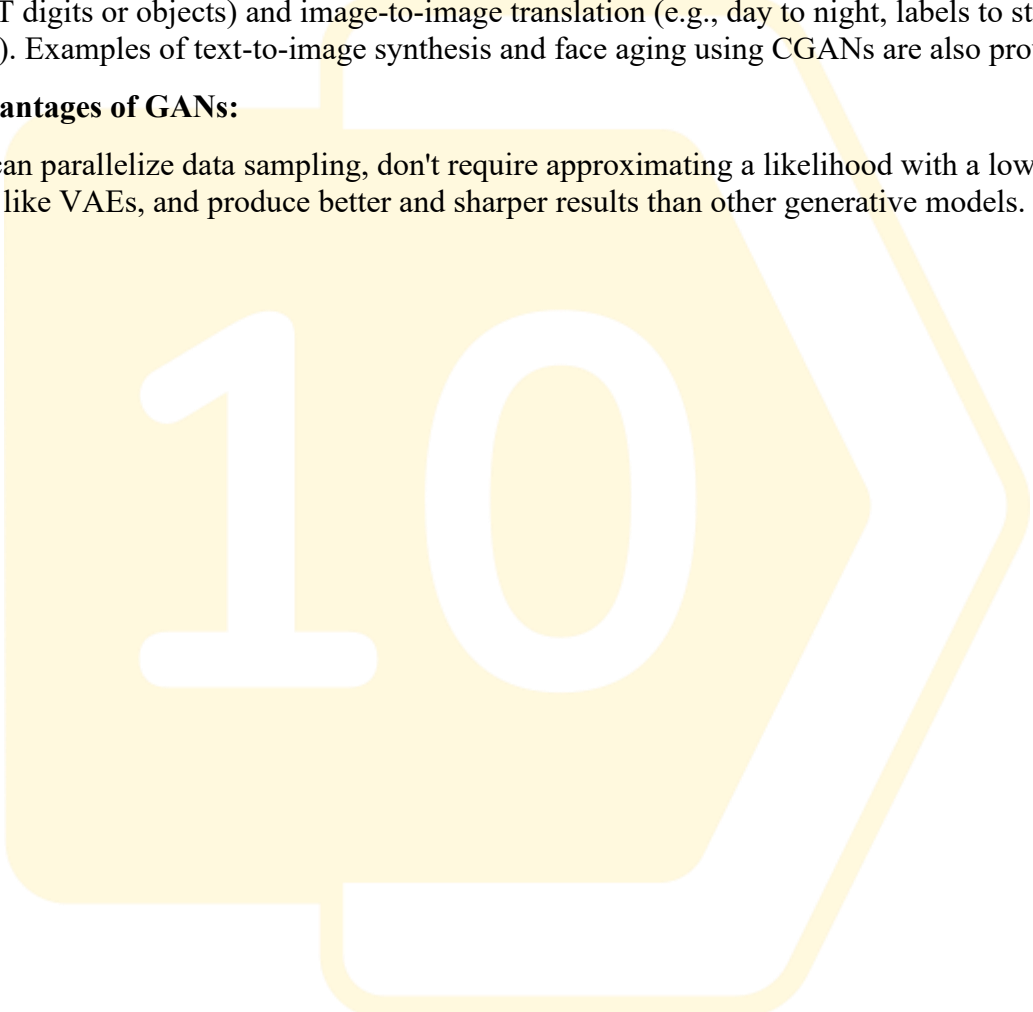
A wide range of applications are presented, such as high-quality image generation, image inpainting, super-resolution, video prediction, facial attribute manipulation, and image/text-to-image translation.

- **Conditional GANs (CGANs):**

These extend GANs by conditioning both the generator and discriminator on additional information y (e.g., class labels). This allows for targeted image generation (e.g., specific MNIST digits or objects) and image-to-image translation (e.g., day to night, labels to street scenes). Examples of text-to-image synthesis and face aging using CGANs are also provided.

- **Advantages of GANs:**

They can parallelize data sampling, don't require approximating a likelihood with a lower bound like VAEs, and produce better and sharper results than other generative models.



10